AI Applications

Laboratory Guide - Part II

CrewAI Cloud Deployment, REST API Authentication, and Flask Integration

Laboratory type: Guided, hands-on tutorial

Estimated time: 2-3 hours, excluding deployment delays

Prerequisite: Part I - Local CrewAI Deployment and Flask Web Application

Scope: Cloud deployment, API testing, Flask integration, and secure GitHub publication. Local setup from Part I is not repeated.

1. Laboratory Overview

In Part I, students deployed a CrewAI workflow locally and created a Flask application that accepts a meeting transcript and generates a Business Requirements Document (BRD). In Part II, students deploy the same workflow to CrewAI Cloud, communicate through authenticated REST requests, and adapt Flask to support local and cloud execution.

Learning Objectives

- Explain differences between local and cloud deployment.
- Deploy an existing CrewAI workflow through CrewAI Studio.
- Identify and test REST endpoints; authenticate using a Bearer Token.
- Manage API credentials using environment variables.
- Launch asynchronous executions, poll for status, and parse JSON.
- Extract generated BRDs and integrate cloud execution into Flask.
- Publish code securely with Git and GitHub.

Prerequisites

- Working CrewAI project and Flask upload page from Part I.
- Python, uv, VS Code, and access to a CrewAI account with deployment capability.
- LLM provider credentials if required by the workflow.
- Basic Python, HTTP, JSON, and environment-variable knowledge.

Cost note: Cloud capabilities, subscription requirements, and prices may change. Verify access and costs before running workflows.

2. Understanding the Architecture

2.1 Local execution

Browser → Flask → Local CrewAI workflow → LLM provider → BRD → Browser. Flask loads the workflow from a local flow.json file.

```
from crewai_cli.run_declarative_flow import load_declarative_flow

def execute_crewai_local(transcript):
    flow = load_declarative_flow(FLOW_PATH)
    result = flow.kickoff(inputs={"transcript": transcript})
    return str(result)
```

2.2 Cloud execution

Browser → Flask → CrewAI Cloud REST API → Cloud agents → BRD → Flask → Browser. Flask remains local; only the agent workflow is hosted remotely.

Feature	Local	Cloud
Workflow runs	Local computer	CrewAI-managed cloud
Access method	Python function	REST API
Authentication	LLM credentials as needed	Cloud Bearer Token
Results	Python return value	JSON API response
Monitoring	Local logs	Status endpoint

Discussion: Does deploying CrewAI to the cloud mean the entire application is cloud-hosted? No. Flask still runs locally in this laboratory.

3. Deploy the Workflow to CrewAI Cloud

3.1 Open and test the workflow

- Sign in to CrewAI Studio and open the workflow from Part I.
- Verify workflow configuration and run a Studio test.
- Confirm the workflow generates a BRD before deploying.

3.2 Deploy

- Locate the deployment function in CrewAI Studio.
- Follow deployment prompts and wait for completion.
- Record the generated deployment URL, such as https://YOUR-DEPLOYMENT.crewai.com.

Checkpoint 1: Record the deployment URL and confirm the deployment is healthy.

3.3 API endpoints verified in our deployment

Method	Endpoint	Purpose
GET	/healthcheck	Deployment health
GET	/inputs	Required workflow inputs
POST	/kickoff	Start execution
GET	/status/{kickoff_id}	Retrieve status and results

Students should verify the endpoints exposed by their own deployment.

3.4 Health check

Open https://YOUR-DEPLOYMENT.crewai.com/healthcheck in a browser. A healthy deployment may return:

```
{"status": "healthy", "version": "1.15.28"}
```

Additional fields may be present. The root URL may return {"detail":"Not Found"} because the service exposes an API rather than a homepage.

4. Configure API Authentication

A Bearer Token is included in the HTTP Authorization header. Treat it as confidential:

```
Authorization: Bearer YOUR_PRIVATE_TOKEN
```

4.1 Obtain and store the token

- Locate deployment credentials in CrewAI Studio.
- Copy the Bearer Token associated with the deployment.
- Never paste real tokens into source code, screenshots, or GitHub.

4.2 Configure .env

```
CREWAI_CLOUD_URL=https://YOUR-DEPLOYMENT.crewai.com
CREWAI_BEARER_TOKEN=YOUR_PRIVATE_TOKEN
CREWAI_EXECUTION_MODE=cloud
```

Keep existing local-execution variables as needed. In the tested deployment, the Bearer Token successfully authenticated /inputs.

4.3 Configure .gitignore

```
.env
.env.*
!.env.example
.venv/
__pycache__/
*.pyc
.crewai/
output/
```

A safe .env.example may contain placeholder variable names, never actual secrets.

4.4 Install requests

```
uv add requests
```

5. Experiment 1 - Test API Authentication

Create test_crewai_cloud.py:

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()
BASE_URL = os.environ["CREWAI_CLOUD_URL"].rstrip("/")
token = os.environ["CREWAI_BEARER_TOKEN"]
headers = {"Authorization": f"Bearer {token}"}
response = requests.get(f"{BASE_URL}/inputs", headers=headers, timeout=30)
print("HTTP Status:", response.status_code)
print("Response:", response.text)
```

```
response.raise_for_status()
```

Run:

```
uv run python test_crewai_cloud.py
```

For our workflow, the successful response was:

```
HTTP Status: 200
Response: {"inputs":["transcript"]}
```

The result confirms both authentication and the required input name. Checkpoint 2: Demonstrate HTTP 200 from /inputs.

HTTP code	Typical meaning
200	Request succeeded
401	Missing or invalid authentication
403	Insufficient permission
404	Incorrect endpoint or URL
500	Server-side problem

6. Experiment 2 - Initiate Cloud Execution

The /kickoff endpoint accepts a JSON object containing the required transcript input and returns an execution identifier.

```
{"inputs": {"transcript": "Meeting transcript goes here."}}
```

6.1 Create test_crewai_kickoff.py

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()
BASE_URL = os.environ["CREWAI_CLOUD_URL"].rstrip("/")
token = os.environ["CREWAI_BEARER_TOKEN"]
headers = {"Authorization": f"Bearer {token}"}
payload = {"inputs": {"transcript": (
    "The insurance company needs a digital policy change request system. "
    "Customers must submit requests online. Staff must review and approve "
    "requests. The system must maintain an audit trail."
)}}
response = requests.post(
    f"{BASE_URL}/kickoff", headers=headers, json=payload, timeout=30
)
print("HTTP Status:", response.status_code)
print("Response:", response.text)
response.raise_for_status()
```

Run:

```
uv run python test_crewai_kickoff.py
```

Expected response structure:

```
{"kickoff_id":"EXAMPLE-EXECUTION-ID"}
```

Important: HTTP 200 means the kickoff request succeeded, not that the BRD is finished. Each kickoff starts a new execution and may incur charges. Checkpoint 3: Record the kickoff_id.

7. Experiment 3 - Monitor Execution

Long-running workflows separate execution initiation from result retrieval. The client repeatedly requests GET /status/{kickoff_id} until completion. This technique is called polling.

7.1 Create test_crewai_status.py

```
import os
import json
import requests
from dotenv import load_dotenv

load_dotenv()
BASE_URL = os.environ["CREWAI_CLOUD_URL"].rstrip("/")
token = os.environ["CREWAI_BEARER_TOKEN"]
kickoff_id = "REPLACE_WITH_YOUR_KICKOFF_ID"
headers = {"Authorization": f"Bearer {token}"}
response = requests.get(
    f"{BASE_URL}/status/{kickoff_id}", headers=headers, timeout=30
)
print("HTTP Status:", response.status_code)
response.raise_for_status()
data = response.json()
print(json.dumps(data, indent=2, ensure_ascii=False))
```

Run:

```
uv run python test_crewai_status.py
```

7.2 Interpret actual response structure

The tested deployment returned top-level keys: state, status, flow_id, result, result_json, usage_metrics, last_step, last_executed_task, last_event, events, events_count, events_by_type, progress, triggered_by, execution_origin, results, and source.

A completed execution showed state = SUCCESS while status = None. Therefore this deployment uses state, not status, to detect completion.

```
print("Execution state:", data.get("state"))
result = data.get("result")
if isinstance(result, dict):
    print("Result fields:", list(result.keys()))
```

The result dictionary contained raw, pydantic, agent_role, usage_metrics, messages, tool_failures, plan, todos, replan_count, and last_replan_reason. The generated Markdown BRD was a string in result["raw"].

```
if data.get("state") == "SUCCESS":
    result = data.get("result") or {}
    raw = result.get("raw")
    if isinstance(raw, str):
```

```
print("\nGenerated BRD:")
print(raw)
```

Checkpoint 4: Retrieve the generated BRD without starting another paid execution.

8. Integrate CrewAI Cloud with Flask

Keep the local execution function and add a cloud execution function. Make a backup of app.py before editing. Add imports os, time, and requests. Ensure load_dotenv() runs before reading environment variables.

8.1 Cloud URL configuration

```
CREWAI_CLOUD_URL = os.environ.get("CREWAI_CLOUD_URL", "").rstrip("/")
```

8.2 Implement execute_crewai_cloud

```
def execute_crewai_cloud(transcript):
    """Execute a CrewAI workflow through CrewAI Cloud."""
    token = os.getenv("CREWAI_BEARER_TOKEN")
    if not token:
        raise RuntimeError("CREWAI_BEARER_TOKEN is missing")
    if not CREWAI_CLOUD_URL:
        raise RuntimeError("CREWAI_CLOUD_URL is missing")

    headers = {"Authorization": f"Bearer {token}"}
    response = requests.post(
        f"{CREWAI_CLOUD_URL}/kickoff",
        headers=headers,
        json={"inputs": {"transcript": transcript}},
        timeout=30,
    )
    response.raise_for_status()
    kickoff_id = response.json()["kickoff_id"]
    print("Kickoff ID:", kickoff_id)

    deadline = time.monotonic() + 300
    while time.monotonic() < deadline:
        status_response = requests.get(
            f"{CREWAI_CLOUD_URL}/status/{kickoff_id}",
            headers=headers,
            timeout=30,
        )
        status_response.raise_for_status()
        data = status_response.json()
        state = data.get("state")
        print("Execution state:", state)

        if state == "SUCCESS":
            result = data.get("result") or {}
            raw = result.get("raw")
            if not isinstance(raw, str):
                raise RuntimeError("No text BRD returned")
            return raw
        if state in ("FAILED", "FAILURE", "ERROR", "CANCELLED"):
            raise RuntimeError(f"Cloud execution failed: {state}")
        time.sleep(5)

    raise TimeoutError(
        "Cloud execution exceeded the polling limit. "
        f"Kickoff ID: {kickoff_id}"
    )
```

The function authenticates, submits a transcript, obtains a kickoff ID, polls every five seconds, checks the execution state, and returns result["raw"]. A polling timeout does not necessarily stop the remote execution.

8.3 Preserve local mode

```
def execute_crewai_local(transcript):
    flow = load_declarative_flow(FLOW_PATH)
    result = flow.kickoff(inputs={"transcript": transcript})
    return str(result)
```

8.4 Select execution mode

```
def execute_crewai(transcript):
    mode = os.getenv("CREWAI_EXECUTION_MODE", "local").lower()
    if mode == "cloud":
        return execute_crewai_cloud(transcript)
    if mode == "local":
        return execute_crewai_local(transcript)
    raise ValueError(f"Unsupported execution mode: {mode}")
```

The existing Flask /run and /upload handlers can continue calling execute_crewai(transcript). Set CREWAI_EXECUTION_MODE=cloud or local in .env and restart Flask to switch.

9. End-to-End Flask Test

- Run uv run python app.py.
- Open http://127.0.0.1:5000/.
- Upload a .docx meeting transcript and click Generate BRD.
- Observe kickoff ID and execution-state messages in the terminal.
- Confirm the completed BRD appears in the browser.

```
Kickoff ID: EXAMPLE-EXECUTION-ID
Execution state: RUNNING
Execution state: RUNNING
Execution state: SUCCESS
```

Intermediate state names may vary. Checkpoint 5: Demonstrate upload, cloud execution, and BRD display.

Architectural limitation: The current Flask handler waits synchronously for cloud execution. Long requests may time out. A production-oriented design should return the kickoff ID immediately and let the browser poll a separate Flask status endpoint. Never expose the Bearer Token in browser JavaScript.

10. Troubleshooting

Problem	Possible cause / action
Root URL returns Not Found	API service; try /healthcheck
Not authenticated / 401	Verify Authorization header and token
403	Check credential permissions
404	Verify deployment URL and endpoint
Missing token	Check .env and load_dotenv()
/kickoff fails	Inspect JSON payload and HTTP response
Execution incomplete	Check /status and cloud logs
result["raw"] missing	Inspect result and result_json structure
Browser waits too long	Synchronous request; consider asynchronous polling
Local works, cloud fails	Test individual cloud API scripts

Debug in order: health → authentication → /inputs → /kickoff → /status → BRD extraction → Flask integration. This isolates errors and avoids repeated paid runs.

11. GitHub and Security

- Initialize Git and configure .gitignore.
- Review staged files for credentials and confidential data.
- Stage and commit through VS Code Source Control.
- Publish initially to a private GitHub repository.
- Verify .env, virtual environment, and confidential transcripts are excluded.

Suggested commit message: Add CrewAI Cloud API integration to Flask application.

```
Digital-Policy-Change-Request-System/
  .gitignore
  .env.example
  README.md
  app.py
  pyproject.toml
  uv.lock
  test_crewai_cloud.py
  test_crewai_kickoff.py
  test_crewai_status.py
  src/
  templates/index.html
```

Checkpoint 6: Demonstrate the GitHub repository without publishing credentials.

12. Student Exercises

Exercise 1 - API Verification

Run the authentication, kickoff, and status scripts independently. Record HTTP codes and explain each endpoint.

Exercise 2 - JSON Analysis

Identify the execution state, result, usage metrics, and progress fields. Explain why state, not status, is used in this deployment.

Exercise 3 - Local versus Cloud

Run the same transcript in both modes. Compare execution time, BRD completeness, requirements accuracy, error handling, and maintenance. Distinguish normal LLM variability from deployment effects.

Exercise 4 - User Interface

Add progress or execution-status messages to the webpage.

Optional Advanced Exercise - Asynchronous Integration

Return kickoff_id immediately from Flask; have the browser poll a Flask status route; let Flask query CrewAI Cloud; show progress and display the final BRD. Keep the Bearer Token server-side.

13. Deliverables and Grading

- GitHub repository link.
- Working Flask source code and three API test scripts.
- Screenshot of successful cloud execution and BRD in browser.
- Short technical report on architecture, authentication, asynchronous execution, and local/cloud comparison.

Component	Points
Successful cloud deployment	15
Authentication and endpoint testing	20
Kickoff, status, and BRD extraction	20
Flask integration	25
GitHub and credential protection	10
Report and reflection	10
TOTAL	100

14. Questions for Reflection

- Why does /kickoff return an identifier instead of the BRD?
- How is HTTP 200 different from workflow state SUCCESS?
- Why should a Bearer Token be stored in .env?
- What are the tradeoffs of local and cloud execution?
- Why is synchronous polling unsuitable for some production applications?
- How would you support simultaneous users?
- What additional security controls would public deployment require?

15. Conclusion

Students extended a local CrewAI application into a cloud-integrated AI system. They deployed a workflow, authenticated REST requests, initiated asynchronous executions, monitored status, extracted BRDs, integrated the workflow with Flask, and practiced secure source control.

Key principle: An AI workflow becomes more useful when exposed through a well-defined interface and integrated into an application that addresses a practical need.

End of Laboratory Guide - Part II